CAPSTONE PROJECT 1

Proposal

Creole LLM: A Model-Agnostic SKILL.md Plugin for Structured Context Compression in AI Coding Agents

by

CHEW ZHAN HONG

23018039

BSc (Hons) in Computer Science

Supervisor: Dr Sathishkumar Veerappampalayam Easwaramoorthy

Semester: April 2026

Date: 9 October, 2026

School of Computing and Artificial Intelligence

Faculty of Engineering and Technology

Sunway University

Table of Contents

Abstract *3*

Chapter 1: Introduction *5*

1.1 Background Study 5

1.2 Problem Statement 7

1.3 Objectives and Scope / Limitations 8

1.4 Research Significance 9

Chapter 2: Literature Review *12*

2.1 Context Window Constraints and the Inference Efficiency Problem 12

2.2 Context Engineering as an Emerging Research Field 13

2.3 Surveys of the Prompt Compression Landscape 14

2.4 Soft Prompt Compression Methods 15

2.4.1 AutoCompressors 16

2.4.2 Nano-Capsulator 17

2.5 Hard Prompt Compression Methods 18

2.5.1 Selective Context 18

2.5.2 Dynamic Attention-Aware Compression (DAC) 19

2.6 Task-Agnostic and Reinforcement Learning-Based Approaches 20

2.7 Token Reduction as a Quality Intervention 21

2.8 Research Gap and Positioning of Creole LLM 22

Chapter 3: Methodology *24*

3.1 Research Design and Approach 24

3.2 System Architecture 24

3.3 Compression Trigger Design 26

3.4 Anchor and Transient Classification Protocol 27

3.5 Edge Case Handling 30

3.6 Tools and Technologies 32

3.7 Evaluation Design 32

3.8 Computing Factors and Constraints 34

Chapter 4: Work Plan and Timeline *36*

References *54*

Abstract

AI coding agents re-read the entire session on every inference pass. On a short session that barely matters. On a long one, the context window fills up, inference gets slower and more expensive, and the session eventually breaks mid-task. Developers using Claude Code, Cursor, Codex, Opencode, or Hermes run into this regularly on complex work like multi-file refactoring or extended debugging. Li et al. [1] put a number on the underlying cause: self-attention scales quadratically with sequence length. Karpathy [2] describes the result at the application level: agents without persistent structured context rediscover the same knowledge from scratch on every query, compounding no understanding across the session.

The compression literature has solutions, but each comes with a cost that most developers cannot absorb. AutoCompressors [3] need a dedicated NVIDIA A100 GPU and model fine-tuning before they can be used. Nano-Capsulator [4] gets 81.4% length reduction but requires eight hours of LoRA training on two A40 GPUs first. Selective Context [5] is training-free but needs a separate language model running at inference time, which does not fit inside a SKILL.md skill file. DAC [6] improves on that with attention-aware metrics but keeps the same infrastructure dependency. RTLIR [7] and TPC [8] both describe plug-and-play deployment but still require training the compression agent before deployment. Two survey papers, Chang et al. [9] and Li et al. [1], both identify a training-free method that matches trained quality as an open research problem. The specific combination of training-free operation, cross-model portability, and SKILL.md compatibility therefore remains unaddressed in the existing literature.

This project proposes Creole LLM, a SKILL.md plugin with a lightweight wrapper that compresses AI coding agent sessions without any of those requirements. Drawing on the adaptive threshold principle of Selective Context [5] and the attention-critical content protection findings of DAC [6], and motivated by the quality reframing of Kong et al. [10], the plugin classifies prior context into anchors comprising the original task statement and current working state, and transient content comprising resolved reasoning chains and redundant dialogue. When the session reaches a user-configured percentage of the model's context window, transient content is collapsed into a structured natural language summary block while anchors are preserved verbatim. The plugin is compatible with any agent supporting skill files as defined by Anthropic [11], [12], [13], including Claude Code, Cursor, Codex, Opencode, and Hermes. Evaluation compares anchor recall between compressed and uncompressed

sessions at equivalent token counts. The project contributes to UN SDG 9 by removing the infrastructure barrier to context compression, and to SDG 4 by making AI coding tools more coherent and accessible for developers regardless of hardware access.

Chapter 1: Introduction

1.1 Background Study

AI coding agents have become a practical part of software development. Tools like Claude Code, Cursor, Codex, Opencode, and Hermes enable developers to conduct extended multi-turn coding sessions in which the agent assists with code generation, debugging, refactoring, documentation, and architectural decision-making across a continuous conversation. The practical utility of these agents depends on their ability to maintain coherent understanding across the full duration of a session, from the initial task specification through to the final deliverable, and this dependency makes context management a first-order concern rather than an optimisation consideration.

The architectural foundation of these agents, the Transformer model, processes inputs through a self-attention mechanism that attends to every token in the context window simultaneously. Li et al. [1] identify this mechanism as having quadratic memory and computational complexity with respect to sequence length, meaning that each doubling of context length quadruples the computational cost of an inference pass. While engineering optimisations such as Flash Attention and key-value caching reduce the constant factor of this cost, the quadratic scaling relationship is a fundamental property of standard self-attention. Chang et al. [9] formalise this challenge in their survey of efficient prompting methods, identifying the computational burden imposed by long prompts as one of the two primary motivations for the entire prompt compression research agenda.

The context window is also finite. Anthropic's platform documentation [11], [12], [13] establishes that the SKILL.md skill file architecture is specifically designed around token budget awareness, with a three-level progressive disclosure mechanism. Metadata loads at startup for roughly 100 tokens. Instruction bodies load on trigger. Reference files load on demand. That hierarchy exists because loading everything at once exhausts the budget too quickly for productive sessions. OpenAI [14] draws the same conclusion independently and recommends proactive compaction before the window fills, not as an advanced technique but as standard practice.

Karpathy [2] frames the same problem from the practitioner angle. Agents without persistent structured context rediscover the same knowledge from scratch on every query. Nothing accumulates. The session grows longer but not smarter, because the model re-reads the full history on every turn with no mechanism to separate what still matters from what has

already been resolved. Mei et al. [15] formalise this in their context engineering survey, placing it within the intra-session context management subfield and identifying it as an active open problem across over 1,400 surveyed papers.

The particular characteristics of AI coding sessions amplify the general context window problem in ways specific to the coding domain. A coding session is not a sequence of independent exchanges but a progressive accumulation of interdependent decisions. The developer specifies a goal, the agent works toward it, decisions accumulate, subtasks complete, and the history grows. Completed reasoning about a function already implemented, superseded approaches rejected in favour of alternatives, and redundant restatements of established constraints all occupy context window tokens without contributing to remaining work. The distinction between operationally relevant and operationally resolved content is more structurally consistent in coding sessions than in general conversational text, a property that Creole LLM exploits.

Unlike human memory, LLM-based agents have no persistent memory outside the context window. Every response is generated from a full re-reading of every prior turn. Karpathy [2] identifies this as a compounding statelessness problem at the practitioner level: agents without persistent structured context produce no quality improvement despite accumulating history, because they rediscover the same knowledge from scratch on every query. Kong et al. [10] identify the underlying mechanism theoretically: softmax attention normalisation means low-relevance tokens suppress the relative attention weight assigned to high-relevance tokens across vision, language, and multimodal architectures, actively degrading output quality.

1.2 Problem Statement

AI coding agents built on large language models suffer from a well-defined architectural inefficiency: on every inference pass, the model re-reads the entire conversation history from the beginning. As a session grows in length, this behaviour consumes an increasing proportion of the available context window, not because new information is being introduced but because previously resolved content continues to occupy space. When the context window fills, the session either truncates older content, losing potentially critical context such as the original task specification, or breaks entirely, forcing the developer to restart and re-establish the working context from scratch.

This problem has two distinct dimensions. The first is computational: the quadratic complexity of self-attention documented by Li et al. [1] means that longer contexts are significantly more expensive per inference pass, with API costs and latency growing substantially as sessions accumulate history. The second is functional: session continuity degrades as the agent loses access to earlier context through truncation, producing responses inconsistent with or unaware of decisions made earlier in the session.

Existing compression approaches address the computational dimension but introduce new barriers. Chevalier et al. [3] demonstrate that AutoCompressors reduce inference memory usage by 36% and inference time by 32% but require fine-tuning the base language model on sequences up to 30,720 tokens using dedicated GPU hardware, rendering compressed representations non-transferable to other model deployments. Chuang et al. [4] address portability through Nano-Capsulator, achieving 81.4% prompt length reduction across Vicuna-13B, PaLM, and Claude2, but the framework requires LoRA fine-tuning taking approximately eight hours on two NVIDIA A40 GPUs before deployment. Li et al. [5] propose Selective Context as a training-free alternative using self-information scores computed by a small causal language model, achieving 50% context reduction with a BERTScore drop of only 0.023, but the requirement for a running language model at inference time is architecturally incompatible with a SKILL.md skill file. Zhao et al. [6] extend this with attention-aware metrics and dynamic entropy recalculation, achieving consistent improvements of 4.03 points over entropy-only baselines on LongBench, but retain the same inference-time model dependency.

Reinforcement learning approaches including RTLIR [7] and TPC [8] seek to achieve task-agnostic portability while retaining compression quality. Huang et al. [7] demonstrate that binary token-level retention decisions learned through reinforcement learning produce an

average 6% accuracy improvement across tasks, while Liskavets et al. [8] show that reward-guided task descriptors allow generalisation across task domains without per-task fine-tuning. Both results are significant contributions to the portability problem, but both methods require training the compression agent before deployment, and neither is compatible with the SKILL.md execution model.

The gap that motivates this project is therefore specific: no existing approach delivers context compression as a portable, training-free, agent-native protocol that can be embedded in a SKILL.md skill file and executed without external tooling, inference-time scoring models, or prior training. Chang et al. [9] identify the compression-quality trade-off for training-free methods as an open problem. Li et al. [1] confirm at NAACL 2025 that the field has not produced a training-free method matching the quality of trained approaches. Creole LLM addresses this gap by exploiting the structured semantic properties of coding sessions specifically, achieving training-free compression quality grounded in session semantics rather than general statistical measures.

1.3 Objectives and Scope / Limitations

This project has three primary objectives. The first is to design a model-agnostic context compression protocol for AI coding agent sessions, formalised as a SKILL.md plugin with a lightweight wrapper, that classifies session context into anchors comprising the original task statement and current working state, and transient content comprising resolved reasoning and redundant dialogue, triggering compression when the session reaches a user-configured percentage of the model's context window. This percentage-based trigger is theoretically motivated by the adaptive threshold principle of Selective Context [5], whose percentile-based filtering adapts to the distribution of the specific context rather than applying universal fixed values. The second objective is to implement the plugin in a form compatible with the SKILL.md specification as documented by Anthropic [11], [12], [13], verifiable across multiple skill-supporting agents including Claude Code, Cursor, Codex, Opencode, and Hermes, with no requirement for model fine-tuning, API modification, or external scoring infrastructure. The third objective is to evaluate the plugin's ability to preserve session coherence after compression, measured by comparing anchor recall between a compressed session and an uncompressed baseline at equivalent token counts.

The scope of this project is bounded to AI coding agents that support skill files as defined by the SKILL.md specification. The compression mechanism operates at the session

level, classifying semantic units based on their functional role in the session structure rather than their statistical properties as individual tokens. The anchor classification draws on the attention-critical content protection principle established by Zhao et al. [6], identifying the original task and working state as the content whose removal would most degrade downstream generation quality for any subsequent turn, consistent with the compression objective formulation of Chang et al. [9]. Evaluation measures anchor recalls as a direct proxy for session coherence and does not extend to full downstream code correctness assessment, task completion rate, or human evaluation of output quality.

Several limitations apply. The compression trigger is a user-configured percentage parameter rather than a learned or dynamically computed threshold, representing a deliberate simplification consistent with the training-free design objective but potentially sub-optimal for sessions with atypical content distributions. The anchor and transient classification relies on the agent's instruction-following capability when executing the SKILL.md protocol, meaning classification accuracy depends on the underlying model's ability to follow structured semantic rules. Sessions with highly interleaved reasoning, in which anchor and transient content cannot be cleanly separated at the turn level, are identified as a limitation and a direction for future work. The evaluation does not capture the quality improvement dimension predicted by Kong et al. [10]; this is identified as a direction for future evaluation.

1.4 Research Significance

The significance of this project operates at three levels: practical accessibility for software developers, academic contribution to the prompt compression literature, and broader implications for computing accessibility in the context of AI tooling.

At the practical level, the inability to maintain long coherent sessions in AI coding agents represents a direct productivity constraint on software developers using these tools. Extended coding tasks such as multi-file refactoring, architecture design, codebase migration, and systematic debugging regularly exceed context window limits with current agent tools when conducted in a single session. Both Anthropic [11], [12], [13] and OpenAI [14] acknowledge this in their platform documentation, with each recommending active context management strategies to extend session coherence. Karpathy [2] further illustrates the practical severity, observing that without persistent structured context, agents produce no quality improvement from accumulated session history because they rediscover the same knowledge from scratch on every query. Creole LLM provides a concrete implementation of

these recommended strategies as a portable, training-free plugin requiring no configuration beyond specifying a window percentage threshold.

At the academic level, this project contributes a training-free alternative within a literature that has not resolved the portability-performance tension without either accepting training requirements or accepting inference-time scoring overhead. Chang et al. [9] identify this tension as the central challenge in prompt compression. Li et al. [1] confirm it remains unresolved as of NAACL 2025. The design of Creole LLM draws on the theoretical foundations of Selective Context [5] in their percentile-based adaptive filtering framework, DAC [6] in demonstrating that attention-critical content does not correlate with entropy, RTLIR [7] in framing token retention as an optimisation problem, and Kong et al. [10] in their quality reframing of token reduction. Combining these foundations into a rule-based semantic implementation contributes a methodological approach that extends the insights of the established literature into a deployment context that no existing method addresses.

At the level of computing accessibility, the significance of a training-free approach extends beyond its immediate application domain. AutoCompressors [3] require a dedicated NVIDIA A100 GPU with 80 GB memory for fine-tuning. Nano-Capsulator [4] requires two NVIDIA A40 GPUs with 48 GB memory each. RTLIR [7] and TPC [8] require training infrastructure for their compression agents. These requirements effectively exclude the majority of individual developers and small teams from the benefits of context compression. A single SKILL.md file deployable without any training infrastructure directly addresses this accessibility barrier. Kong et al. [10] establish that the benefits of token reduction extend to output quality improvement and not only to computational efficiency, reinforcing that this accessibility gap carries functional consequences as well as economic ones. Karpathy [2] observes that agents operating without persistent structured context rediscover the same knowledge from scratch on every query regardless of session length, a compounding inefficiency that disproportionately affects developers relying on limited context windows without access to sophisticated memory infrastructure.

This project contributes to United Nations Sustainable Development Goal 9: Industry, Innovation and Infrastructure, specifically Target 9.b on supporting domestic technology development in developing countries and Target 9.c on increasing access to information and communications technology. The core contribution of Creole LLM, a training-free context compression plugin requiring no GPU infrastructure, directly reduces the barrier to accessing

AI coding infrastructure for developers in resource-constrained environments. The project additionally contributes to SDG 4: Quality Education, specifically Target 4.4, which calls for increasing the number of people with relevant technical skills for employment and entrepreneurship. AI coding agents are increasingly central to software developer education and professional practice; improving their coherence and session length through accessible tooling supports the quality of AI-assisted learning and development environments available to developers regardless of institutional affiliation or hardware access.

Chapter 2: Literature Review

2.1 Context Window Constraints and the Inference Efficiency Problem

Start with the architecture. Transformer models compute self-attention by comparing every token against every other token in the input. For a sequence of n tokens, that produces an n-by-n attention matrix. Memory scales as O(n-squared). So does the number of floating-point operations required to compute it. Li et al. [1] identify this quadratic growth as the root driver of inference cost in long-context deployments, and Chang et al. [9] name it as one of the two primary motivations for prompt compression research. Neither paper offers a way around it within standard self-attention. The math is the math.

For a coding agent on a short session, this barely matters. For one running into tens of thousands of tokens, it matters a lot. Double the session length and the inference cost per pass quadruples, not doubles. Flash Attention and grouped query attention reduce the constant factor but not the growth rate. So an extended session becomes progressively more expensive not because the current task is harder, but simply because the history has grown. At some point the context window fills, and then something worse happens: either the oldest content is cut to make room, or the session breaks entirely.

Anthropic [11], [12], [13] treat this as a design constraint, not an edge case. The three-level progressive disclosure architecture in the SKILL.md specification exists because token budgets need active management. Metadata loads at startup for roughly 100 tokens. Instruction bodies load on trigger. Reference files load on demand. That hierarchy did not appear because it is elegant. It appeared because loading everything at once exhausts the budget too quickly for productive sessions. OpenAI [14] draws the same conclusion independently and recommends proactive compaction before the window fills, not as an advanced technique but as standard practice.

Karpathy [2] frames the same problem from the practitioner angle. Agents without persistent structured context rediscover the same knowledge from scratch on every query. Nothing accumulates. The session grows longer but not smarter, because the model re-reads the full history on every turn with no mechanism to separate what still matters from what has already been resolved. Mei et al. [15] formalise this in their context engineering survey, placing it within the intra-session context management subfield and identifying it as an active open problem across over 1,400 surveyed papers.

Kong et al. [10] make the point that matters most for how Creole LLM positions its contribution. Across vision, language, and multimodal architectures, removing redundant tokens does not merely cut cost. It improves output quality. Softmax attention normalises weights across the full window, so irrelevant tokens dilute the attention available to relevant ones. Cut the irrelevant tokens and the model attends more strongly to what remains. Compression is not a trade-off between cost and quality. Done correctly, it improves both.

Coding sessions have a property that makes the problem tractable in a way general conversation does not. Every turn relates to the same underlying task. The developer specifies a goal, the agent works toward it, decisions accumulate, subtasks complete, and the history grows. What remains operationally necessary throughout all of that is consistent: the original task specification and the current implementation state. What stops being necessary after a completed subtask is also consistent: the reasoning that led to decisions already implemented. Li et al. [1] and Mei et al. [15] both identify domain-specific compression as underexplored. Karpathy [2] arrives at the same conclusion from practice, noting that a well-designed schema file specifies exactly what to maintain and what to treat as resolved. That structural distinction is the foundation of Creole LLM.

2.2 Context Engineering as an Emerging Research Field

Mei et al. [15] spent considerable effort making the field legible. Their synthesis of over 1,400 papers establishes Context Engineering as a named discipline with defined subfields, and the taxonomy they produce is useful for locating exactly where Creole LLM sits relative to everything else.

The six subfields they identify are context retrieval, context processing, context management, context memory, context demonstration, and multi-agent context coordination. Retrieval covers how relevant information is sourced from external databases or prior records. Processing covers how raw context is filtered and compressed. Management covers how context is maintained and updated within a running session. Memory covers persistence across sessions. Demonstration covers few-shot example selection. Coordination covers shared context in multi-agent systems.

Creole LLM sits in context management, not retrieval. That distinction matters. Retrieval-augmented approaches, which Karpathy [2] describes as rediscovering the same knowledge from scratch on every query, address a different problem: bringing external information into a new session. Context management addresses what to do with information

that has already been generated within the session currently in progress. Creole LLM compresses the current session's history. It does not retrieve from external sources and it does not persist across sessions.

Within context management, Mei et al. [15] identify two primary strategies: truncation and summarisation. Truncation removes the oldest content when the window fills. In a coding session that usually means losing the original task specification, which is the worst possible content to lose. Summarisation replaces a portion of context with a compact representation. That preserves more information but requires a mechanism to generate the summary, which typically introduces latency and overhead. Anthropic [12] explicitly warns against adding unnecessary token overhead in SKILL.md skill file design.

Creole LLM differs from both. It does not truncate. It does not produce a generic summary. It classifies content based on its functional role in the session, preserves anchor content verbatim based on the attention-critical protection principle of Zhao et al. [6], and replaces transient content with a structured summary in the natural language format validated by Chuang et al. [4]. The classification is the novel part. Neither truncation nor standard summarisation distinguish between anchor and transient content at the session level.

Karpathy [2] describes a structurally similar distinction in his wiki maintenance pattern: the schema file must specify what facts to preserve across queries and what to treat as resolved. The anchor and transient classification in Creole LLM is the coding session equivalent of that schema. Mei et al. [15] note that the field lacks approaches combining within-session context management with domain-specific structural knowledge. Creole LLM is specifically that.

2.3 Surveys of the Prompt Compression Landscape

Two survey papers provide the conceptual vocabulary for everything that follows in this chapter. Chang et al. [9] cover efficient prompting broadly. Li et al. [1] focus on compression specifically. Between them they define the space, identify the open problems, and make the gap that Creole LLM addresses explicit.

Chang et al. [9] organise compression into two output formats. Text-to-Vector methods convert natural language into dense continuous representations, trading portability for compression density. Text-to-Text methods produce shorter natural language, trading some compression density for portability across any model that processes standard text. Creole LLM is a Text-to-Text method. That choice is not incidental. The portability of natural language

output is what makes cross-agent compatibility possible, as Chuang et al. [4] demonstrate with Nano-Capsulator's performance across Vicuna-13B, PaLM, and Claude2 without retraining.

The formal objective Chang et al. [9] set out is to minimise the KL divergence between the output distributions of the original and compressed prompts, subject to an informativeness threshold. In practice that means identifying content whose removal most increases the divergence and keeping it; while discarding content whose removal has minimal effect. For a coding session, the content that most increases divergence when removed is the original task and the working state, because every subsequent response is conditioned on both. That is why Creole LLM classifies them as anchors.

Li et al. [1] arrive at a specific finding that carries direct weight for this project: as of NAACL 2025, no training-free method matches the compression quality of trained approaches on general text. They name this as an open problem. They also note that most compression research targets general text benchmarks and that domain-specific approaches remain underexplored. Both observations apply here. Creole LLM does not compete on general text benchmarks. It claims domain-specific quality in coding sessions, where the structural properties of the session make rule-based classification viable in a way that general text does not support.

Chang et al. [9] also flag three challenges shared by all compression methods. First, there are no ground-truth labels for which content is dispensable, since information value depends on the task. Creole LLM handles this by exploiting coding session structure rather than relying on general labels. Second, removing content that participates in referential chains can break discourse coherence. Creole LLM handles this by replacing transient content with a structured summary that maintains logical continuity. Third, cross-study comparison is difficult without standardised benchmarks. Creole LLM defines anchor recall as a concrete task-specific metric to address this directly.

2.4 Soft Prompt Compression Methods

Soft prompt compression methods convert natural language into continuous vector representations, which the downstream model attends to as soft prompt prefixes instead of the original text. Chang et al. [9] classify all of them as Text-to-Vector, noting that their outputs are not portable across models with different embedding geometries. Li et al. [1] acknowledge that they achieve the strongest compression performance at high ratios but flag the model-

specificity as a fundamental limitation. Two methods in this category are directly relevant: AutoCompressors [3] and Nano-Capsulator [4].

2.4.1 AutoCompressors

Chevalier et al. [3] build AutoCompressors by adding special summary tokens to the input vocabulary of a pre-trained language model, whose output embeddings become the summary vectors. Training uses an unsupervised cross-entropy objective: the model processes randomly segmented documents and predicts each segment's tokens conditioned on concatenated summary vectors from all previous segments. No labels specify what to retain. The model learns what is useful for predicting subsequent content.

The key design decision is summary accumulation. Earlier memory architectures passed only the most recent summary to the current segment. AutoCompressors concatenate all prior summaries, giving every segment direct access to the full compressed history. Ablation experiments confirm this matters: on held-out perplexity evaluation with three compressed segments, summary accumulation produces 5.6% improvement over the fine-tuned OPT-2.7B baseline, compared to 4.3% for the RMT baseline without it. The gap grows with more compressed segments.

The numbers are good. Fine-tuning on OPT-2.7B and Llama-2-7B on sequences up to 30,720 tokens on a single NVIDIA A100 GPU with 80 GB memory, the method achieves a 40:1 compression rate per segment at 50 summary vectors. On in-context learning across 11 benchmarks, summary vectors from 750-token demonstration segments outperform plain-text demonstrations of equivalent length on 8 of 11 tasks.

The problem is the embedding space. Summary vectors are specific to the fine-tuned model instance that generated them. An AutoCompressor trained from OPT-2.7B produces vectors in OPT-2.7B's embedding space. Those vectors mean nothing to Llama-2, to Claude, or to any API-accessible model, because the embedding geometries differ. Chevalier et al. [3] acknowledge this in their limitations section. For a tool that needs to work across Claude Code, Cursor, Codex, Opencode, and Hermes without modification, this is a non-starter. The SKILL.md execution model documented by Anthropic [11], [12], [13] operates on natural language text. There is no mechanism to pass embedding vectors through it.

2.4.2 Nano-Capsulator

Chuang et al. [4] start from the same problem Chevalier et al. [3] leave unsolved: portability. Their solution is to make the compressed output natural language rather than embedding vectors. Nano-Capsulator trains a compression model using two concurrent objectives: a semantic preservation loss that minimises the distance between hidden state embeddings of the original and compressed prompts, and a reward function that evaluates downstream task performance and penalises outputs exceeding the target length.

The semantic preservation loss works through replication and summarisation. The compression model is prompted to replicate the original, generating a reference embedding, and to summarise it into a shorter Capsule Prompt. The loss minimises mean squared error between the two resulting embeddings. The reward function adds task performance signal by comparing the downstream model's accuracy on the original versus the Capsule Prompt, penalising compression that loses task-relevant content. Together these produce shorter natural language that the downstream model can use.

The results justify the complexity. Across CommonsenseQA, GSM8K, MultiRC, and TriviaQA-Long, Nano-Capsulator achieves 81.4% length reduction for chain-of-thought compression while maintaining performance across Vicuna-13B, PaLM, and Claude2 without retraining the compression model for any of them. API cost savings on Claude2 range from 63.9% on GSM8K to 80.1% on TriviaQA-Long. On CommonsenseQA with PaLM, the Capsule Prompt outperforms the original uncompressed prompt at 75.5% versus 73.7%, which fits the quality improvement pattern Kong et al. [10] describe.

But the training cost is significant. Eight hours on two NVIDIA A40 GPUs with 48 GB memory each for chain-of-thought tasks. Four hours for reading comprehension. That must happen before deployment. The SKILL.md architecture Anthropic [11], [12], [13] documents are specifically designed for zero-setup deployment: metadata at startup, instruction bodies on trigger, reference files on demand. A plugin that requires multi-GPU training before it can be used contradicts that design principle at the most basic level. Nano-Capsulator also requires running a separate LoRA fine-tuned compression model before each agent inference pass. A SKILL.md skill file runs within the agent's own forward pass. These are architecturally incompatible.

2.5 Hard Prompt Compression Methods

Hard prompt compression methods keep natural language as both input and output, identifying and removing low-information content to produce a shorter but coherent version of the original. Because their outputs are standard text, they work across any model that processes standard text, making them inherently more portable than soft prompt methods. Chang et al. [9] classify them as Text-to-Text. Li et al. [1] identify them as the category most relevant to training-free deployment. Two methods are directly relevant: Selective Context [5] and DAC [6].

2.5.1 Selective Context

Li et al. [5] build Selective Context around self-information from Shannon's information theory. For a token x at position t, self-information I(x) is the negative log-likelihood of that token given all preceding tokens: I(x) = -log2 P(xt given x0 through x(t-1)). Low self-information means the token was predictable and carries little new content. High self-information means it was surprising and carries more. The method sums self-information across tokens to score phrases and sentences, then filters by percentile rather than fixed threshold, keeping only the units above a specified cutoff.

The percentile-based approach adapts to the actual content distribution of the specific context being compressed, rather than applying a universal fixed cutoff. That is the design principle Creole LLM borrows for its trigger mechanism: a fixed token count threshold does not mean the same thing on an 8,000-token model and a 200,000-token model, so a percentage of the available window adapts to the deployment context the same way Li et al.'s [5] percentile threshold adapts to the content distribution.

The results are solid. At a 50% compression ratio on arXiv papers, BBC News articles, and ShareGPT conversations, Selective Context maintains BERTScore F1 above 0.9 and ROUGE-1 above 0.64 compared to full-context generation. Inference memory drops 36% and inference time drops 32%. Phrase-level filtering outperforms both token-level and sentence-level across all tasks. Faithfulness testing on GPT-3.5 shows only 3.8% unfaithful claims at the 0.5 compression ratio, rising to 5.1% at 0.65.

The limitation is the scoring infrastructure. Li et al. [5] use LLaMA-7B for the LLaMA and Vicuna families and GPT-3 curie for OpenAI models, running them sentence by sentence to avoid the tendency of LLMs to assign lower self-information to later tokens in long sequences. Running a second language model at inference time is not compatible with a

SKILL.md skill file. The skill executes within the agent's own forward pass. There is no slot for an external scoring model. Zhao et al. [6] identify a second limitation: self-information does not predict which tokens receive high attention. An entropy-only method cannot protect the content the model attends to most.

2.5.2 Dynamic Attention-Aware Compression (DAC)

Zhao et al. [6] build DAC to fix the two things wrong with entropy-only compression. The first problem: entropy and attention are almost unrelated. The second problem: entropy values shift when you remove tokens, so a one-shot compression policy computed from initial entropy values makes increasingly bad decisions as it goes.

They establish the first problem quantitatively. Analysing 200 sequences from NarrativeQA using Qwen2-7B, they find a Pearson correlation of 0.095 between accumulated attention scores and information entropy. Near zero. That means high-attention tokens are distributed more or less randomly across the entropy distribution, so an entropy-based filter removes a substantial fraction of the most attended tokens by chance. They confirm the practical consequence with an ablation: removing all attention-critical tokens before entropy-based compression produces performance worse than random token removal on single-document QA tasks from LongBench. That is not a marginal degradation. Entropy-only compression actively harms performance in this condition.

They establish the second problem through visualisation. At a 0.9 compression ratio, removing just 10% of tokens causes significant entropy shifts in many of the tokens that remain, because a token's self-information depends on what precedes it. Remove the preceding token and the current one looks more surprising than it actually was. The Pearson correlation between pre-compression and post-compression entropy falls below 0.5 when the compression ratio exceeds 0.5. A static one-pass policy fails at high compression ratios because the entropy landscape it was computed from no longer exists.

DAC addresses both with an integrated metric and a dynamic procedure. The metric fuses information entropy and accumulated attention scores additively: (1 minus alpha) times entropy plus alpha times attention score per token. The optimal alpha through experiments on single-document QA from LongBench is 0.8, meaning attention gets four times the weight of entropy. The dynamic procedure divides compression into multiple stages, recalculating entropy with a small language model at each stage to account for the shifts caused by prior

removals. A consecutive compression constraint prevents removing adjacent tokens in the same stage.

On LongBench at a 0.5 compression ratio, DAC improves 4.03 points over Selective Context and surpasses LLMLingua-2 by 1.33 points on average. On GSM8K at the same ratio, DAC achieves 74.37% exact match, compared to 61.33% for Selective Context and 75.21% for the uncompressed baseline. The ablation confirms every component contributes, and experiments on LLaMA3 series models confirm the gains are architecture independent.

The 0.095 correlation finding is the key number for Creole LLM. In a coding session, the original task and working state are the content that receives the most consistent attention across all turns: every response is conditioned on both. Their self-information in a mature session is low, because they consist of well-established formulations the model predicts confidently. An entropy-based filter would remove them. Creole LLM protects them by design through the anchor classification, without measuring attention at runtime. The functional outcome is the same as DAC's [6] attention-critical protection, implemented as a semantic rule rather than a runtime measurement. DAC still requires a running language model for its entropy computation. Creole LLM does not.

2.6 Task-Agnostic and Reinforcement Learning-Based Approaches

A third line of compression research frames token retention as a learned policy problem. Chang et al. [9] note it as an emerging direction. Li et al. [1] place it within parameter-efficient fine-tuning and synthetic language mechanism categories. Two recent papers in this space, TPC [8] and RTLIR [7], are worth examining in detail because both explicitly target the same deployment properties as Creole LLM while stopping short of achieving them.

Liskavets et al. [8] train a compression agent that uses context-aware sentence embeddings to represent each sentence's semantic relationship to its surrounding context, and a reward-guided task descriptor that estimates what task is being performed without an explicit query. The reward signal comes from the downstream model's output quality when given the compressed versus original prompt. After training, the agent is deployed without any downstream-model-specific adaptation, achieving task-agnostic portability. The results generalise across task domains. The sentence embedding computation at inference time requires an embedding model. The compression agent requires training. Neither is compatible with SKILL.md.

Huang et al. [7] go token-level. The state is the current retained token set. The action is a binary retain or discard decision for each token. The reward comes from downstream generation quality. The agent learns through interaction with the downstream model. Once trained, it is described as plug-and-play and LLM-agnostic. Across benchmarks it achieves an average 6% accuracy improvement over uncompressed baselines, which fits the quality improvement finding of Kong et al. [10]. The plug-and-play framing refers to post-training deployment. The training itself requires infrastructure.

Both TPC [8] and RTLIR [7] identify zero-setup deployment as a desirable property and describe their post-training behaviour as plug-and-play. Neither achieves zero training cost. That gap is where Creole LLM sits. The RL framing of RTLIR [7] also clarifies what Creole LLM is doing semantically: the optimal retention policy for a coding session maximises downstream quality given the retained set. The anchor classification is a deterministic approximation of that policy for coding sessions specifically, derived from structural properties of the domain rather than learned from data. It is tractable because coding sessions have consistent structure. The RL agent learns because general text does not.

2.7 Token Reduction as a Quality Intervention

Kong et al. [10] reframe the entire motivation for compression research. Most of the literature treats compression as an efficiency measure that accepts some quality loss in exchange for lower cost. Kong et al. [10] argue that framing is wrong in a class of cases that matters.

The mechanism they identify is softmax attention normalisation. Attention weights sum to one across all tokens in the window. Add low-relevance tokens and you reduce the weight available to high-relevance tokens, because the total is fixed. Remove the low-relevance tokens and the high-relevance ones receive more. The result is higher output quality, not just lower cost. Kong et al. [10] demonstrate this across vision, language, and multimodal Transformer architectures, finding consistent evidence that removing redundant tokens improves quality rather than merely preserving it.

For Creole LLM this changes the contribution claim. The project does not trade session length against response quality. It claims that compressing resolved, redundant session history improves the quality of responses to subsequent turns by concentrating attention on anchor content. The original task and working state, already the highest-attention content in the session

by the structural argument from Zhao et al. [6], receive more attention weight once the transient content diluting them is removed.

Karpathy [2] arrives at the same conclusion from practice without the theoretical framing. Agents without structured persistent context produce no quality improvement from accumulated history. The reason is exactly the attention dilution mechanism Kong et al. [10] identify history that has not been structured accumulates noise rather than signal, and the model attends to all of it equally on every pass. Creole LLM's compression is not a concession to limited context windows. It is an active quality intervention.

2.8 Research Gap and Positioning of Creole LLM

Three dimensions define the gap this project addresses. Every method reviewed in this chapter fails on at least one of them.

The first is training cost. AutoCompressors [3], Nano-Capsulator [4], RTLIR [7], and TPC [8] all require GPU training before deployment. The hardware requirements range from one NVIDIA A100 to two A40s over multiple hours. Selective Context [5] and DAC [6] are training-free but need running language models at inference time. Chang et al. [9] identify the training-portability trade-off as the central challenge. Li et al. [1] confirm at NAACL 2025 that it is still unresolved.

The second is domain specificity. Every method in this review targets general text. None are designed for coding agent sessions specifically. The structural properties of coding sessions, specifically the consistent presence of an original task as the highest-attention content, the identification of completed subtasks as natural compression boundaries, and the predictable accumulation of resolved reasoning as the primary redundant content, are available for exploitation but go unused in general-purpose approaches. Karpathy [2] describes exactly these properties in his discussion of what a schema file should specify. Li et al. [1] and Mei et al. [15] both identify domain-specific compression as underexplored.

The third is SKILL.md compatibility. The specification documented by Anthropic [11], [12], [13] defines a protocol for deploying reusable agent behaviours without external infrastructure. A compression plugin that works within this architecture must run inside the agent's own forward pass, using natural language instructions the agent follows natively. No existing method does this. Soft prompt methods need separate model instances. Selective

Context [5] and DAC [6] need external scoring models. RL-based methods [7], [8] need trained agents. SKILL.md compatibility eliminates all of them.

Creole LLM addresses all three. It is training-free, grounded in the percentile-based adaptive trigger of Selective Context [5] and the attention-critical anchor protection of DAC [6]. It is domain-specific, exploiting the structural properties that Karpathy [2], Mei et al. [15], and Li et al. [1] independently identify. It is SKILL.md-compatible, encoding compression as a semantic classification protocol the agent executes natively. The quality reframing of Kong et al. [10] positions it as an improvement rather than a trade-off. Table 2.1 sets out the comparison across all reviewed methods and confirms that Creole LLM is the only approach achieving the target outcome across all seven dimensions.

Table 2.1: Comparison of prompt compression approaches across key design dimensions

Method	Training Required	Cross-Model Portable	NL Output	SKILL.md Compatible	Agent-Native	Zero Infrastructure
AutoCompressors [3]	Yes	No	No	No	No	No
Nano-Capsulator [4]	Yes	Yes	Yes	No	No	No
Selective Context [5]	No	Yes	Yes	Partial	No	Partial
DAC [6]	No	Yes	Yes	Partial	No	Partial
TPC [8]	Yes	Yes	Yes	No	No	No
RTLIR [7]	Yes	Yes	Yes	No	No	No
Creole LLM (This project)	No	Yes	Yes	Yes	Yes	Yes

Chapter 3: Methodology

3.1 Research Design and Approach

This chapter covers the methodology behind Creole LLM: why it is built the way it is, how it works, and how it gets evaluated. Four decisions sit at the centre of it. The choice of SKILL.md as the delivery mechanism. The design of the anchor and transient classification protocol. The percentage-based trigger. And the evaluation design measuring anchor recall. Each one traces back to a specific finding from Chapter 2, not to intuition or trial and error.

The research design is empirical and artefact-based. The output is the plugin itself, a SKILL.md file plus a lightweight wrapper. Every component in it comes from a theoretical principle established in the compression literature rather than from tuning or guesswork. The evaluation is comparative: run the plugin against a baseline and see whether it actually preserves what it claims to preserve.

Chang et al. [9] classify compression along two dimensions, the mechanism and the output format. Creole LLM is Text-to-Text, producing natural language output, because that is what stays portable across models, as Chuang et al. [4] demonstrate with Nano-Capsulator working across Vicuna-13B, PaLM, and Claude2 without retraining. The mechanism is rule-based semantic classification, not statistical scoring and not learned optimisation. Three reasons. Statistical scoring needs a running language model at inference time, which does not fit inside SKILL.md. Training-based methods need GPU infrastructure, which contradicts the zero-infrastructure goal. And rule-based classification actually works for coding sessions specifically, because the structural properties of a coding session create a consistent line between anchor and transient content without needing to learn it from data. Li et al. [1] flag this domain-specific tractability as underexplored. This project tests whether it holds up.

3.2 System Architecture

Creole LLM has two pieces: the SKILL.md plugin file and a lightweight wrapper. The plugin file holds the compression protocol itself, written as natural language instructions the agent follows during a normal turn. The wrapper holds configuration, compatibility declarations, and reference files the plugin pulls in on demand. This split comes straight from the SKILL.md specification Anthropic [11], [12], [13] documents.

The plugin runs inside the three-level loading structure Anthropic [12] describes. Metadata loads at startup for roughly 100 tokens. The full instruction body loads only when

the trigger fires, so it costs nothing on turns where compression is not needed. Reference files load on demand during the compression pass itself. Creole LLM respects this hierarchy deliberately. Adding overhead that eats into the compression benefit would defeat the point.

Karpathy [2] describes a schema file pattern, something like CLAUDE.md or AGENTS.md, that turns a generic agent into a disciplined maintainer of structured knowledge by telling it exactly what to track and how to update it. Creole LLM's SKILL.md instruction body does the same job for compression: what counts as anchor content, what counts as transient, how to build the summary, how to carry the anchor representation forward across multiple compression passes in the same session. So the schema-as-protocol idea here has two legs under it, the academic SKILL.md specification and the practitioner pattern Karpathy [2] describes independently.

The wrapper does two things. First, it exposes the threshold parameter, a percentage of the context window, defaulting to 60%, justified by the adaptive threshold logic from Li et al. [5]. Second, it declares platform compatibility. Anthropic [11] confirms SKILL.md works on Claude Code, the Claude API, and Claude.ai. The wrapper extends that to Cursor, Codex, Opencode, and Hermes. Running compression inside the agent's own forward pass rather than as a separate process matches what OpenAI [14] recommends for context management generally: keep it native, not bolted on.

Fig. 3.1 lays out how these pieces connect inside the agent runtime, from the incoming session turn through the trigger check, the three-level plugin loading structure, and the classification and summary generation stages, back to the compressed context returned to the session.

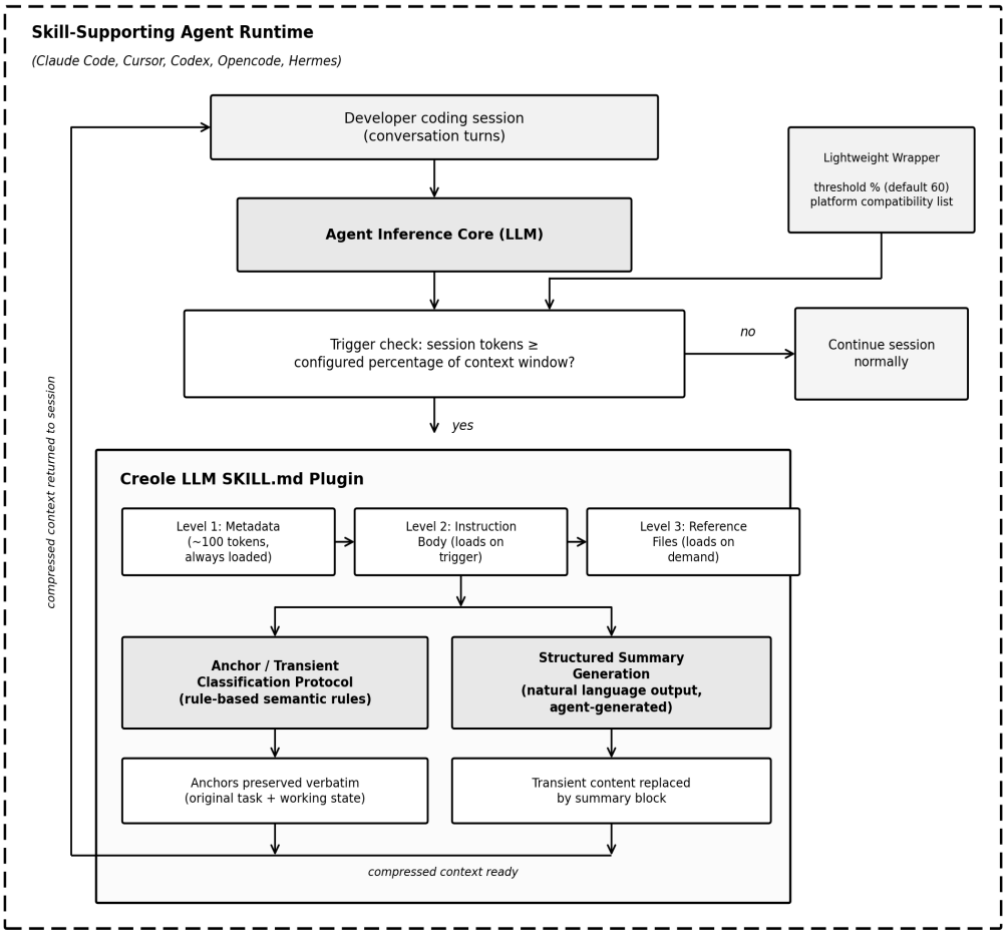


Fig. 3.1. System architecture of the Creole LLM SKILL.md plugin, showing the three-level progressive disclosure structure and the compression pipeline within the agent runtime.

3.3 Compression Trigger Design

The trigger fires as a percentage of the context window. Once session tokens hit the configured percentage of the model's max window, the plugin loads and compression starts. Default is 60%.

Why percentage and not a fixed count? Li et al. [5] answer this for filtering specifically, with their percentile-based threshold that adapts to the actual content distribution instead of applying a flat cutoff everywhere. Creole LLM applies the same logic one level up, at the trigger. A fixed 10,000-token threshold is 50% of a 20,000-token window but only 5% of a 200,000-token one. Same number, completely different meaning depending on the model. A

60% threshold means the same thing regardless of window size: compress when you're a bit more than halfway full, every time.

60% specifically comes from two places. Mei et al. [15] find that proactive context management beats reactive: act before the window is nearly full, not after. 60% leaves 40% of the window as buffer. OpenAI [14] makes the same recommendation from the platform side, specifically to avoid losing the original task and early decisions to truncation if the window is allowed to fill completely. Developers can adjust the threshold, which matters because what counts as a sensible default varies by model size and typical session length, consistent with the adaptive philosophy of Li et al. [5].

There is also a secondary trigger tied to subtask completion. Finishing a subtask is a clean semantic boundary: everything before it becomes transient, the working state moves forward. If a subtask completes on a turn that also crosses the percentage threshold, compression fires immediately rather than waiting for the next turn. Mei et al. [15] identify completed subtasks as exactly this kind of natural boundary for context management.

3.4 Anchor and Transient Classification Protocol

The classification step is the core of the whole protocol: split prior context into anchors and transient content. This is where Li et al. [5], Zhao et al. [6], and Chang et al. [9] actually get implemented, as rules rather than as scoring functions.

Anchors are the content whose removal would hurt the most, in the Chang et al. [9] sense of maximally increasing the divergence between compressed and uncompressed output distributions. Two things consistently qualify in coding sessions. The original task: the developer's goal, the codebase context, the explicit constraints. Every response from that point forward depends on it. Remove it and the agent loses its reference point entirely. The current working state: what has actually been built and decided so far, including any active unresolved issue. Remove it and the agent forgets where the implementation stands and starts repeating or contradicting itself.

This goes back to Zhao et al. [6] directly. Their 0.095 correlation between attention and entropy means scoring by entropy alone would not reliably protect this content. The original task and working state often have low self-information in a mature session, since they are well-established phrasing the model predicts confidently, but they get attended to constantly because every single response depends on them. Classifying them as anchors and protecting them

outright, regardless of entropy, is the rule-based version of what Zhao et al. [6] accomplish through measurement. Karpathy [2] makes basically the same point from practice: a schema file has to say explicitly what to preserve, because left alone, agents treat everything in context as equally important.

Transient content is everything else. Resolved reasoning chains, debugging exchanges for errors already fixed, earlier drafts of code superseded by a final version, repeated statements of constraints already captured elsewhere. None of it conditions future output anymore. Its presence is exactly the attention dilution problem Kong et al. [10] describe: it spreads the model's attention across content that no longer matters, taking weight away from the anchors.

Transient content gets replaced with a structured summary, not deleted outright. The summary has three parts: the task restated concisely, the current working state with implementation progress and open issues, and a one-line note on the most recently completed subtask for continuity. This format follows the natural language approach Chuang et al. [4] validate with Nano-Capsulator, except here the agent generates it itself from the actual session content rather than relying on a separately trained compressor, which is exactly the training step Chuang et al. [4] cannot avoid. The instructions guiding what goes into the summary stay conservative, following the same logic Li et al. [5] apply to filtering: only keep what minimally preserves the output distribution, nothing extra.

Table 3.1 lays the criteria side by side. The dividing line between the two columns is not arbitrary; it maps directly onto the divergence argument from Chang et al. [9] and the attention-versus-entropy finding from Zhao et al. [6] discussed above.

Fig. 3.2 traces the full protocol end to end, from the trigger check through classification, summary generation, the prior-summary integration check, and back to the reassembled compressed context that the session continues from.

TABLE 3.1. ANCHOR VERSUS TRANSIENT CLASSIFICATION CRITERIA

Criterion	**Anchor Content**	**Transient Content**
Definition	Original task statement and current working state	Resolved reasoning, completed exchanges, redundant restatements

Divergence impact [9]	Removal maximally increases output distribution divergence	Removal has minimal effect on output distribution
Attention profile [6]	Consistently high attention across all subsequent turns	Attention weight decays once the subtask is resolved
Self-information	Often low in a mature session (well-established phrasing)	Variable; not a reliable classifier on its own
Compression action	Preserved verbatim, never modified	Replaced with structured natural language summary
Example	“Build a REST API for user auth with JWT”	Debugging exchange for an error already fixed

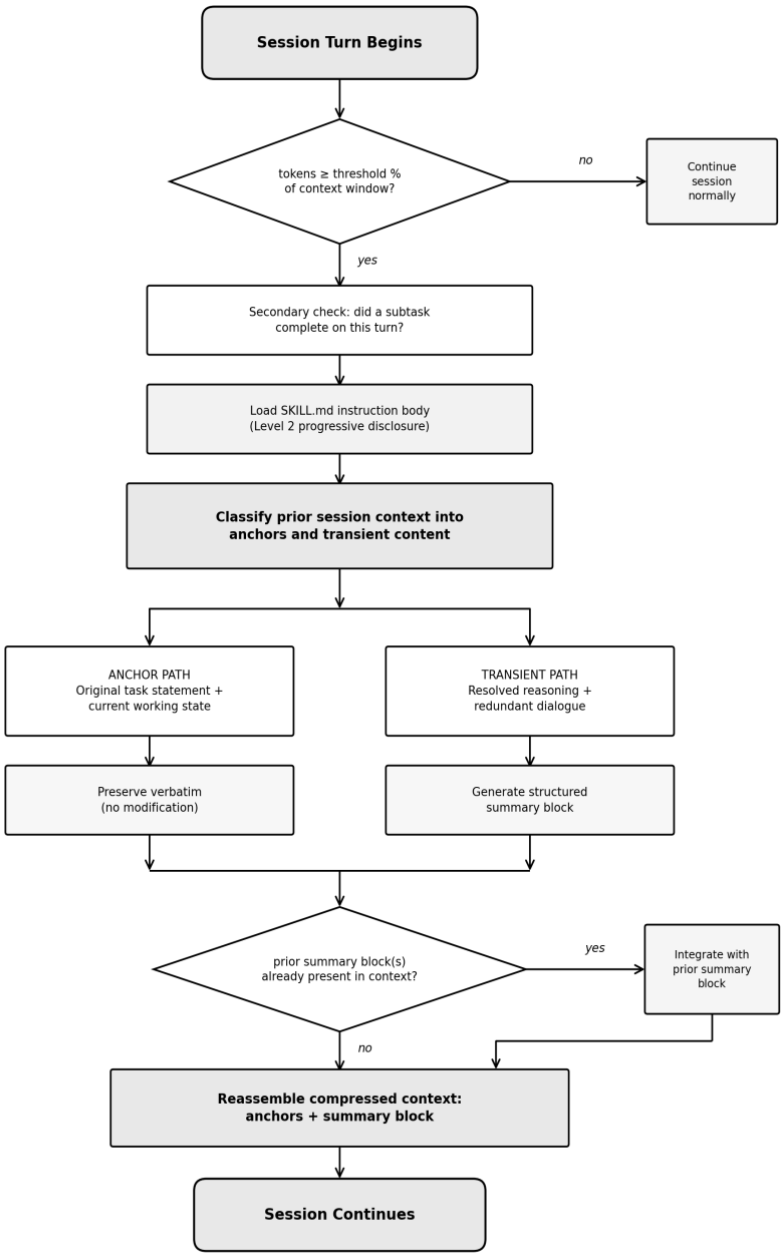


Fig. 3.2. Compression protocol flowchart, from trigger evaluation through classification, summary generation, prior-summary integration, and reassembly of the compressed context.

3.5 Edge Case Handling

Three edge cases needed explicit handling.

First, sessions where the task is not stated in one message but built up across several early turns as the developer refines what they want. Here the protocol has the agent synthesise

those turns into a single anchor statement rather than picking one turn and calling it the task. That is the abstractive approach Mei et al. [15] describe, generating a representation from the content rather than extracting it verbatim.

Second, sessions juggling multiple subtasks at once, which happens whenever a developer switches between debugging one thing and building another in the same conversation. The protocol lists every active subtask in the working state anchor instead of just the most recent one, so nothing gets dropped because the conversation moved on to something else temporarily.

Third, sessions that have already gone through one or more compression passes. If a prior summary block is still sitting in context alongside newer turns, the protocol folds the old summary into the new one rather than treating it as transient and discarding it. The old working state becomes the starting point for the update. This way compression passes build on each other across a long session instead of each one wiping out what the last one captured.

Table 3.2 summarises the trigger condition and protocol response for each of the three cases.

TABLE 3.2. EDGE CASE HANDLING SUMMARY

Edge Case	Trigger Condition	Protocol Response
Distributed task statement	Task specified across multiple early turns rather than one message	Synthesise all relevant turns into a single consolidated anchor statement
Concurrent subtasks	Developer is working on more than one subtask in the same session	Enumerate every active subtask in the working-state anchor with individual status
Prior compression pass	A summary block from an earlier compression already exists in context	Fold the prior summary into the new one; do not treat it as transient

3.6 Tools and Technologies

Three layers of tooling are involved here: the plugin itself, the evaluation infrastructure, and the platforms it runs on.

The plugin follows the SKILL.md schema Anthropic [11], [12], [13] documents: YAML frontmatter for metadata, Markdown for the instruction body, plain text for reference files. Frontmatter carries the plugin name, version, description, compatibility list, and the threshold parameter. The instruction body carries the actual protocol, split into trigger evaluation, classification, and summary generation. This schema-as-protocol approach lines up with the wiki maintenance pattern Karpathy [2] describes, where a schema file governs how an agent maintains structured knowledge over time.

Anthropic [11], [12], [13] confirm SKILL.md works on Claude Code, the Claude API, and Claude.ai. Cursor, Codex, Opencode, and Hermes round out the supported list. Cross-platform testing on at least two distinct agents checks that the protocol runs consistently regardless of which model is underneath, which is the whole point of the portability claim against model-specific approaches like AutoCompressors [3].

For evaluation, a session runner builds reproducible coding sessions with a known structure: a defined task, a sequence of implementation turns that produce a predictable working state, and transient turns that pile up resolved reasoning around the core work. The baseline runs the identical script without triggering compression at all. Comparing anchor recall between the two at matched token counts is a direct test of whether compression preserves what an uncompressed session would still have at the same point, following the comparison method Li et al. [5] and Zhao et al. [6] both use in their own evaluations.

3.7 Evaluation Design

Anchor recall is the headline metric. It measures how well the agent reproduces the original task and the current working state when asked directly, right after a compression pass, compared to how an uncompressed session at the same token count answers the same question.

Three phases. First, establish ground truth: run the uncompressed script to its full length and record what the task and working state actually are. Second, run the compressed condition, trigger at the threshold, continue for a fixed number of turns, then ask the agent to state the task and describe the working state. Score the response against ground truth on completeness,

accuracy, and specificity. Third, run the baseline at the same token count and record the same scores for comparison.

Completeness checks whether every essential piece is present, the goal, the constraints, the implementation progress, any open issues. Accuracy checks whether what is recalled actually matches the ground truth rather than contradicting it. Specificity checks whether the answer is detailed enough to be useful rather than vague filler. Each scored on a three-point scale, averaged across both anchor types and all three dimensions. Table 3.3 lays out the scoring bands for each dimension.

Fig. 3.3 lays out the full three-phase evaluation pipeline: the shared session template branching into the ground truth, compressed, and baseline runs, each proceeding through its own column of steps. The ground truth column terminates as the fixed reference standard that both scoring columns are measured against, while the compressed and baseline columns converge into the final anchor recall comparison.

The main hypothesis: compressed sessions land within a small tolerance of baseline recall at matched token counts. Li et al. [5] set the precedent for what counts as acceptable tolerance, accepting a 0.023 BERTScore drop as fine at 50% compression. A second hypothesis, following from Kong et al. [10], is that compressed sessions might actually beat baseline recall in sessions heavy on transient content, because stripping it out concentrates attention on the anchors that remain. Token savings get measured alongside recall, since the whole point is to show that compression buys real efficiency without giving up coherence.

TABLE 3.3. ANCHOR RECALL ASSESSMENT RUBRIC

Dimension	**Score 1 (Low)**	**Score 2 (Partial)**	**Score 3 (Full)**
Completeness	Most essential elements missing	Some essential elements present	All essential elements present
Accuracy	Recalled content contradicts ground truth	Minor inconsistencies with ground truth	Fully consistent with ground truth
Specificity	Vague or generic description	Somewhat detailed, lacks specifics	Detailed enough to be operationally useful

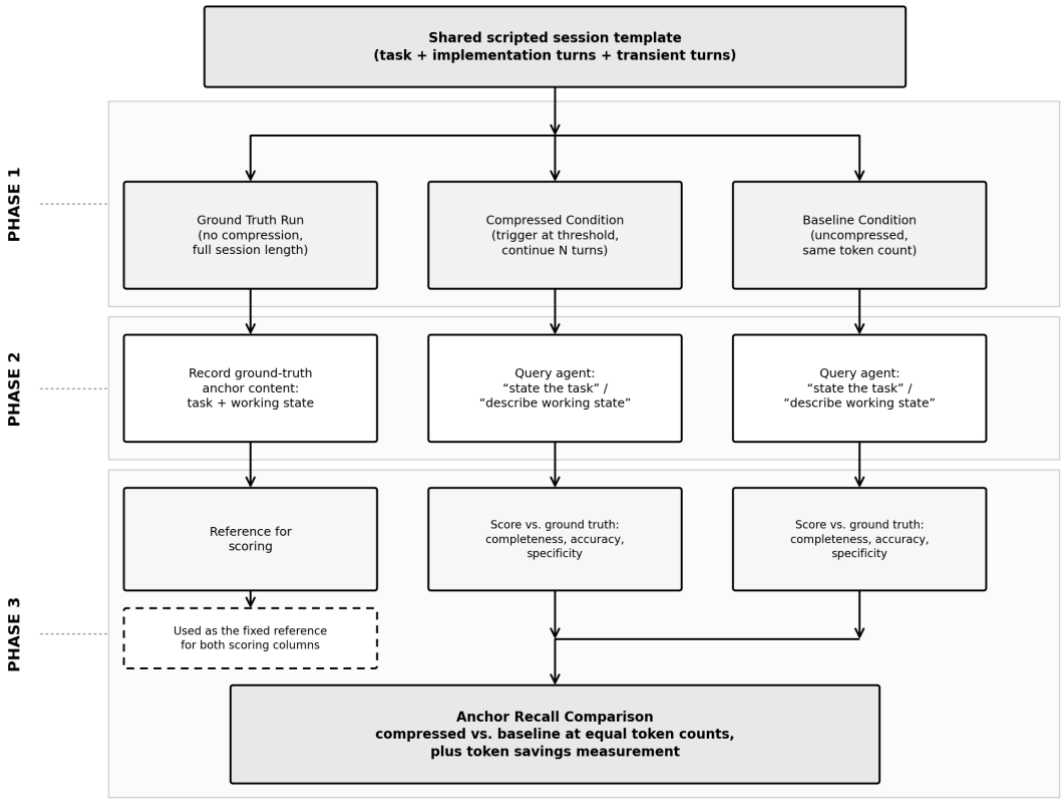


Fig. 3.3. Three-phase evaluation methodology comparing ground truth, compressed, and baseline conditions at matched token counts.

3.8 Computing Factors and Constraints

Four factors shaped how this got built: performance, usability, security, and ethics.

Performance first. Every compression pass costs something: loading the instruction body, running the classification, generating the summary. Anthropic [12] recommends keeping skill instruction bodies under 5,000 tokens for this reason, and Creole LLM follows that. One compression pass costs one extra turn. Against that, the token savings from removing transient content should be substantially larger for any session with a typical mix of anchor and transient content, so the net effect should be positive for anything but very short sessions.

Usability mattered from the start, tied directly to the accessibility argument in Section 1.4 and the SDG 9 framing. No machine learning background needed, no GPU, no training step. The only thing a developer sets is the threshold percentage, which requires no knowledge of specific token counts. 60% works fine as a default across most contexts without adjustment,

matching the drop-in deployment goal RTLIR [7] also aims for. This directly answers the accessibility gap from Chapter 2: AutoCompressors [3] need an A100. Nano-Capsulator [4] needs two A40s and eight hours. Selective Context [5] needs a running language model. Creole LLM needs a skill-supporting agent, which is already the baseline requirement for using it at all.

Two security considerations came up. First, the summary block should never carry sensitive content, credentials, API keys, anything personal that showed up earlier in the session. Anthropic [12] recommends exactly this kind of explicit data handling instruction for skills that process session content. Second, the fixed-schema summary format with named fields limits how much room there is for injected content to manipulate later responses.

On ethics, transparency was the priority. Every compression pass produces a visible summary the developer can actually read and check, unlike AutoCompressors [3], where the summary vectors are opaque and unreadable by design. To guard against losing anchor content by mistake, the classification rules lean toward keeping ambiguous content rather than discarding it, prior summaries get folded forward instead of dropped, and the anchor recall evaluation catches systematic misclassification if it happens. The underlying principle is that the developer stays in control and can see what the protocol is doing at every step, consistent with the responsible AI development guidelines documented by Anthropic [11].

Chapter 4: Work Plan and Timeline

Capstone Project Logbook Week 1

Student Information

Name:	Chew Zhan Hong
Student ID:	23018039
Programme:	BSc (Hons) in Computer Science
Project Title:	Creole LLM: A Model-Agnostic SKILL.md Plugin for Structured Context Compression in AI Coding Agents
Supervisor:	Dr Sathishkumar Veerappampalayam Easwaramoorthy

Weekly Entry Template

Week Number:	1
Date Range:	11th of May 2026 to 15th of May 2026

Objectives for the Week:
Gather resources and information to support the entire problem statement, methodology, and other relevant things for the capstone 1 project.
Verify the feasibility of the entire project topic through available scholarly articles within 2023 to 2025.

Activities Completed:
Gathered in total of 10 references coming from scholarly articles, academic papers, and research papers within the years of 2023 to 2025.
Verified each references relevancy and usefulness for the project topic.
Gathered 4 different official documentations from Anthropic, OpenAI, and such about Effective Prompt Engineering, Context Engineering, Skills authoring, and building skills for AI Agents.

Challenges Encountered (if any):
Mostly challenges on the direction of the project and if it is feasible by one person since there were a lot of scholarly articles that mentioned building their own complex frameworks which did make me think about if it was possible for me build this project effectively.

Solutions / Actions Taken (if any):
Mostly read more about the project and its field since a lot of it is still quite new and read more into semantics for prompt and context engineering as that is one of the fundamentals in solving this problem.

Progress Reflection:
For week 1, I have only made progress in gathering resources, solidifying the project topic, verifying the resources and other materials to make sure it is feasible for one person to complete. Sadly there were many doubts about my own abilities mostly from me since even though I do use other people's skills plugins on a regular basis for my other projects, I was not really that familiar with the work that goes into each plugin and the proper semantics and rules that make the AI Agent and LLM consistently efficient in most sessions. However, after reading a lot more this week I did managed to get a more solid fundamental understanding behind the mechanics, rules, methodologies and such used for each specific skills plugin which made me appreciate the work that goes into those plugins and how much thought that needs to go into making those plugins.

Plans for Next Week:

Continue to research and submit the finalized project topic.
Schedule proper bi-weekly meetings with my Supervisor.

Supervisor's Feedback (if any):

Supervisor's Signature | Student's Signature

References

Anthropic. (2025). *Agent skills overview*. Claude API Documentation. https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview

Anthropic. (2025). *Skill authoring best practices*. Claude API Documentation. https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices

Anthropic. (2026). *The complete guide to building skills for Claude*. https://resources.anthropic.com/hubfs/The-Complete-Guide-to-Building-Skill-for-Claude.pdf

Chang, K., Xu, S., Wang, C., Luo, Y., Liu, X., Xiao, T., & Zhu, J. (2024). Efficient prompting methods for large language models: A survey. *arXiv*. https://doi.org/10.48550/arxiv.2404.01077

Chevalier, A., Wettig, A., Ajith, A., & Chen, D. (2023). Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 3829–3846). Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.emnlp-main.232

Chuang, Y., Xing, T., Chang, C., Liu, Z., Chen, X., & Hu, X. (2024). Learning to compress prompt in natural language formats. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 7756–7767). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.naacl-long.429

Hakim, M. A. (2026). Brevity constraints reverse performance hierarchies in language models. *arXiv*. https://doi.org/10.48550/arxiv.2604.00025

Kong, Z., Li, Y., Zeng, F., Xin, L., Messica, S., Lin, X., Zhao, P., Kellis, M., Tang, H., & Zitnik, M. (2025). Token reduction should go beyond efficiency in generative models: From vision, language to multimodality. *arXiv*. https://doi.org/10.48550/arxiv.2505.18227

Li, Y., Dong, B., Guerin, F., & Lin, C. (2023). Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 6342–6353). Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.emnlp-main.391

Li, Z., Liu, Y., Su, Y., & Collier, N. (2025). Prompt compression for large language models: A survey. In *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 7182–7195). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.naacl-long.368

Liskavets, B., Roy, S., Ushakov, M., Klibanov, M., Etemad, A., & Luke, S. (2025). Task-agnostic prompt compression with context-aware sentence embedding and reward-guided task descriptor. *arXiv*. https://doi.org/10.48550/arxiv.2502.13374

Mei, L., Yao, J., Ge, Y., Wang, Y., Bi, B., Cai, Y., Liu, J., Li, M., Li, Z., Zhang, D., Zhou, C., Mao, J., Xia, T., Guo, J., & Liu, S. (2025). A survey of context engineering for large language models. *arXiv*. https://doi.org/10.48550/arxiv.2507.13334

OpenAI. (2025). *Prompt guidance*. OpenAI API Documentation. https://developers.openai.com/api/docs/guides/prompt-guidance

Zhao, Y., Li, Z., Zhao, H., Qi, B., & Guoming, L. (2025). DAC: A dynamic attention-aware approach for task-agnostic prompt compression. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics* (pp. 19395–19407). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.acl-long.952

Capstone Project Logbook Week 2

Student Information

Name:	Chew Zhan Hong
Student ID:	23018039
Programme:	BSc (Hons) in Computer Science
Project Title:	Creole LLM: A Model-Agnostic SKILL.md Plugin for Structured Context Compression in AI Coding Agents
Supervisor:	Dr Sathishkumar Veerappampalayam Easwaramoorthy

Weekly Entry Template

Week Number:	2
Date Range:	18th of May 2026 to 22nd of May 2026

Objectives for the Week:
Gather more resources and information to support the entire problem statement, methodology, and other relevant things for the capstone 1 project.
Verify the feasibility of the entire project topic through available scholarly articles within 2023 to 2025.
Start writing the Abstract, Introduction, Literature Review, and Methodology.

Activities Completed:
Gathered a finalized total of 14 Documentation and References for the project report.
Verified each references relevancy and usefulness for the project topic.
Completed Abstract, Introduction, and Literature Review.
Challenges Encountered (if any):
Mostly challenges on how to properly utilize each documentation and references since it did give me a lot of solid ideas on how to approach the implementation of the project.

Solutions / Actions Taken (if any):
Mostly read more and started practicing writing some prototype files to see what works and if it works at all to reducing token usage and optimizing context memory. Some of it did work but not by a lot.

Progress Reflection:
For week 2, I have significant progress in completing most of the report such as Abstract, Introduction, and Literature Review except for the Methodology, and Work Timeline. Currently I have verified a few prototype implementations that worked but not to the extent I wanted it to be but will be tweaking some of it to see if any of the modern hypotheses would make it any better like using aliases for generic paths and incorporating wiki patterns from Karpathy's research.

Plans for Next Week:
Continue to research and write about the project.
Schedule proper bi-weekly meetings with my Supervisor.

Supervisor's Feedback (if any):

Supervisor's Signature



……

Student's Signature

CWEW……

Capstone Project Logbook Week 3

Student Information

Name:	Chew Zhan Hong
Student ID:	23018039
Programme:	BSc (Hons) in Computer Science
Project Title:	Creole LLM: A Model-Agnostic SKILL.md Plugin for Structured Context Compression in AI Coding Agents
Supervisor:	Dr Sathishkumar Veerappampalayam Easwaramoorthy

Weekly Entry Template

Week Number:	3
Date Range:	25th of May 2026 to 29th of May 2026

Objectives for the Week:
Narrow and focus on the basic outcomes needed for this project such as Language Semantics, LLM Wiki Patterns (first coined by Andrej Karpathy) and some minor aspects like Harnesses.

Focus on reviewing the first draft of the report and making sure I have utilised all relevant content and information for this project.

Activities Completed:
Reviewed the current draft and looked at what can be still added into the report for all chapters.

Found a llm-wiki.md abstract document written by Andrej Karpathy on his github page, but it is not an academic journal or article but rather a simple markdown file describing and detailing an abstract concept of using Graph Wiki Patterns for optimizing context memory usage.

Challenges Encountered (if any):
The main challenge for this week is just my schedule as I didn't have much time in scheduling anything like the SV meetings, I am supposed to request but at least the report is almost completed.

Solutions / Actions Taken (if any):
Solutions for now is to just fix my current schedule and hopefully I can fulfill and complete all my backlogged tasks for all my subjects and commitments.

Progress Reflection:
For Week 3, especially when writing this I did forget to send this logbook to my Supervisor as It wasn't confirmed if I would have the same Supervisor for the next week due to the coordinators not being very clear about the process for telling them when or how a student should report to them for securing their own Supervisor for the Capstone 1 Project and I also realised I still have a lot of things to write and complete for my logbook entries. But for now, the report is nearing its completion.
Plans for Next Week:
Focus on what is relevant to my project instead of adding more.
Fix my schedule so I can efficiently do all of my work and tasks.

Supervisor's Feedback (if any):

Supervisor's Signature Student's Signature

Sathishkumar VE

CHEW.....

References

https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f

Capstone Project Logbook Week 4

Student Information

Name:	Chew Zhan Hong
Student ID:	23018039
Programme:	BSc (Hons) in Computer Science
Project Title:	Creole LLM: A Model-Agnostic SKILL.md Plugin for Structured Context Compression in AI Coding Agents
Supervisor:	Dr Sathishkumar Veerappampalayam Easwaramoorthy

Weekly Entry Template

Week Number:	4
Date Range:	1st of June 2026 to 5th of June 2026

Objectives for the Week:
Review and focus on literature review and its contents for all relevant information for the entire capstone 1 project.
Focus on reviewing the first draft of the report and making sure I have utilised all relevant content and information for this project.
Review everything again on the new draft report.

Activities Completed:
Wrote and added more into chapter 1 and chapter 2 and chapter 3 of the new draft report.
Worked on creating a small proof of concept prototype to test the feasibility and usability of this capstone project. It still needs more testing.

Challenges Encountered (if any):
The main challenge for this week is just my schedule as I didn't have much time in scheduling anything like the SV meetings, I am supposed to request but at least the report is almost completed.

Solutions / Actions Taken (if any):
Solutions for now is to just fix my current schedule and hopefully I can fulfill and complete all my backlogged tasks for all my subjects and commitments.

Progress Reflection:
For Week 4, I have made significant progress in understanding and implementing the project and managed to test out some new ideas and concepts which are already written in the report with the same exact references and there is always something new to do and refine each week.

Plans for Next Week:
Focus on what is relevant to my project instead of adding more.
Fix my schedule so I can efficiently do all of my work and tasks.

Supervisor's Feedback (if any):

Supervisor's Signature

Sathishkumar VE

Student's Signature

C.Y.E.W.……

Capstone Project Logbook Week 5

Student Information

Name:	Chew Zhan Hong
Student ID:	23018039
Programme:	BSc (Hons) in Computer Science
Project Title:	Creole LLM: A Model-Agnostic SKILL.md Plugin for Structured Context Compression in AI Coding Agents
Supervisor:	Dr Sathishkumar Veerappampalayam Easwaramoorthy

Weekly Entry Template

Week Number:	5
Date Range:	8th of June 2026 to 12th of June 2026

Objectives for the Week:
Review and update the Abstract, Introduction, and other chapters.
Review everything again on the new draft report.

Activities Completed:
Finalized the new Abstract and Chapter 1.
Worked on utilizing different and more aggressive methods for the SKILL.md plugin.
Challenges Encountered (if any):
The main challenge for this week is just my schedule again as I have to focus on other assignments and also practicing for a cybersecurity competition representing Sunway, which unfortunately meant that I am unable to do much about the Capstone progress for now.

Solutions / Actions Taken (if any):
Solutions for now is to just fix and manage my schedule better and do better planning for the next coming weeks.

Progress Reflection:
For Week 5, I have made significant progress in understanding and implementing the project and managed to test out some new ideas and concepts which are already written in the report with the same exact references and there is always something new to do and refine each week.

Plans for Next Week:
Focus on what is relevant to my project instead of adding more.
Fix my schedule so I can efficiently do all of my work and tasks.

Supervisor's Feedback (if any):

Supervisor's Signature

Sathishkumar VE

……

Student's Signature

CYEW……

Capstone Project Logbook Week 6

Student Information

Name:	Chew Zhan Hong
Student ID:	23018039
Programme:	BSc (Hons) in Computer Science
Project Title:	Creole LLM: A Model-Agnostic SKILL.md Plugin for Structured Context Compression in AI Coding Agents
Supervisor:	Dr Sathishkumar Veerappampalayam Easwaramoorthy

Weekly Entry Template

Week Number:	6
Date Range:	15th of June 2026 to 19th of June 2026

Objectives for the Week:
Review and update the Abstract, Introduction, and other chapters.
Review everything again on the new draft report.

Activities Completed:
Finalized all three chapters in the report.
Worked on utilizing different and more aggressive methods for the SKILL.md plugin.
Challenges Encountered (if any):
The main challenge for this week is just my schedule again as I have to focus on other assignments and also practicing for a cybersecurity competition representing Sunway, which unfortunately meant that I am unable to do much about the Capstone progress for now.

Solutions / Actions Taken (if any):
Solutions for now is to just fix and manage my schedule better and do better planning for the next coming weeks.

Progress Reflection:
For Week 6, I have made significant progress in understanding and implementing the project and managed to test out some new ideas and concepts which are already written in the report with the same exact references and there is always something new to do and refine each week.

Plans for Next Week:
Wait for my supervisor's feedback on the current draft of the report.
Fix my schedule so I can efficiently do all of my work and tasks.

Supervisor's Feedback (if any):

Supervisor's Signature



……

Student's Signature

CHEW……

Capstone Project Logbook Week 7

Student Information

Name:	Chew Zhan Hong
Student ID:	23018039
Programme:	BSc (Hons) in Computer Science
Project Title:	Creole LLM: A Model-Agnostic SKILL.md Plugin for Structured Context Compression in AI Coding Agents
Supervisor:	Dr Sathishkumar Veerappampalayam Easwaramoorthy

Weekly Entry Template

Week Number:	7
Date Range:	29th of June 2026 to 3rd of July 2026

Objectives for the Week:
Convert the finalized report into IEEE Format and rechecked everything thoroughly to make sure it is comprehensible and complete.

Activities Completed:
Completed the new finalized report in IEEE referencing and made sure everything is properly formatted and written for submission.

Challenges Encountered (if any):
The main challenge for this week is just my schedule again as I have to focus on other assignments and also practicing for a cybersecurity competition representing Sunway, which unfortunately meant that I am unable to do much about the Capstone progress for now.

Solutions / Actions Taken (if any):
Solutions for now is to just fix and manage my schedule better and do better planning for the next coming weeks.

Progress Reflection:
For Week 7, I have made significant progress in finalizing the report in IEEE and now I am just waiting on my Supervisor to check and give me proper feedback on the report writing and if it passes the check then I can start doing the project prototype for my Capstone 2.

Plans for Next Week:
Wait for my supervisor's feedback on the current draft of the report.
Fix my schedule so I can efficiently do all of my work and tasks.
Once i get confirmation from my supervisor then I can start working on the project prototype.

Supervisor's Feedback (if any):

Supervisor's Signature

Sathishkumar VE

……

Student's Signature

CYEW……

Capstone Project Logbook Week 8

Student Information

Name:	Chew Zhan Hong
Student ID:	23018039
Programme:	BSc (Hons) in Computer Science
Project Title:	Creole LLM: A Model-Agnostic SKILL.md Plugin for Structured Context Compression in AI Coding Agents
Supervisor:	Dr Sathishkumar Veerappampalayam Easwaramoorthy

Weekly Entry Template

Week Number:	8
Date Range:	13th of July 2026 to 17th of July 2026

Objectives for the Week:
Finalize the new draft diagrams for Chapter 3 Methodology and send it for review feedback from my Supervisor.

Activities Completed:
Completed most of the draft diagrams except for one which needs to be polished and finalized before sending it to my supervisor for review.
Challenges Encountered (if any):
The main challenge for this week is just my schedule again as I have to focus on other assignments and also competing in said cybersecurity competition which my team managed to get top 5. But now my challenge is still my schedule.

Solutions / Actions Taken (if any):
Solutions for now is to just fix and manage my schedule better and do better planning for the next coming weeks.

Progress Reflection:
For Week 8, I have made significant progress in finalizing the diagrams for Chapter 3 Methodology which will be sent to my Supervisor for a review and once The feedback is received I can then put the report aside until submission date.

Plans for Next Week:
Wait for my supervisor's feedback on the current draft of the diagrams.
Fix my schedule so I can efficiently do all of my work and tasks.
Once i get confirmation from my supervisor then I can start working on the project prototype.

Supervisor's Feedback (if any):

Supervisor's Signature

Sathishkumar VE

……

Student's Signature

CHEW……

References

[1] Z. Li, Y. Liu, Y. Su, and N. Collier, "Prompt compression for large language models: A survey," in Proc. Conf. North Amer. Chapter Assoc. Comput. Linguist.: Human Lang. Technol. (NAACL), 2025, pp. 7182–7195. [Online]. Available: https://doi.org/10.18653/v1/2025.naacl-long.368

[2] A. Karpathy, "llm-wiki.md," GitHub Gist, Apr. 2026. [Online]. Available: https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f

[3] A. Chevalier, A. Wettig, A. Ajith, and D. Chen, "Adapting language models to compress contexts," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2023, pp. 3829–3846. [Online]. Available: https://doi.org/10.18653/v1/2023.emnlp-main.232

[4] Y. Chuang, T. Xing, C. Chang, Z. Liu, X. Chen, and X. Hu, "Learning to compress prompt in natural language formats," in Proc. Conf. North Amer. Chapter Assoc. Comput. Linguist.: Human Lang. Technol. (NAACL), 2024, pp. 7756–7767. [Online]. Available: https://doi.org/10.18653/v1/2024.naacl-long.429

[5] Y. Li, B. Dong, F. Guerin, and C. Lin, "Compressing context to enhance inference efficiency of large language models," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2023, pp. 6342–6353. [Online]. Available: https://doi.org/10.18653/v1/2023.emnlp-main.391

[6] Y. Zhao, Z. Li, H. Zhao, B. Qi, and G. Liu, "DAC: A dynamic attention-aware approach for task-agnostic prompt compression," in Proc. 63rd Annu. Meeting Assoc. Comput. Linguist. (ACL), 2025, pp. 19395–19407. [Online]. Available: https://doi.org/10.18653/v1/2025.acl-long.952

[7] G. Huang, Y. Xiao, L. Jiang, M. Yin, and P. Wang, "Beyond prompt engineering: A reinforced token-level input refinement for large language models," in Proc. AAAI Conf. Artif. Intell., vol. 39, no. 22, 2025, pp. 24113–24121. [Online]. Available: https://doi.org/10.1609/aaai.v39i22.34586

[8] B. Liskavets, S. Roy, M. Ushakov, M. Klibanov, A. Etemad, and S. Luke, "Task-agnostic prompt compression with context-aware sentence embedding and reward-guided task descriptor," arXiv:2502.13374 [cs.CL], Feb. 2025. [Online]. Available: https://doi.org/10.48550/arxiv.2502.13374

[9] K. Chang, S. Xu, C. Wang, Y. Luo, X. Liu, T. Xiao, and J. Zhu, "Efficient prompting methods for large language models: A survey," arXiv:2404.01077 [cs.CL], Apr. 2024. [Online]. Available: https://doi.org/10.48550/arxiv.2404.01077

[10] Z. Kong, Y. Li, F. Zeng, L. Xin, S. Messica, X. Lin, P. Zhao, M. Kellis, H. Tang, and M. Zitnik, "Token reduction should go beyond efficiency in generative models: From vision, language to multimodality," arXiv:2505.18227 [cs.LG], May 2025. [Online]. Available: https://doi.org/10.48550/arxiv.2505.18227

[11] Anthropic, "Skill authoring best practices," Anthropic Developer Docs, 2025. [Online]. Available: https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices

[12] Anthropic, "Agent skills overview," Anthropic Developer Docs, 2025. [Online]. Available: https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview

[13] Anthropic, "The complete guide to building skills for Claude," Anthropic, 2026. [Online]. Available: https://resources.anthropic.com/hubfs/The-Complete-Guide-to-Building-Skill-for-Claude.pdf

[14] OpenAI, "Prompt engineering," OpenAI Platform Documentation, 2025. [Online]. Available: https://platform.openai.com/docs/guides/prompt-engineering

[15] L. Mei, J. Yao, Y. Ge, Y. Wang, B. Bi, Y. Cai, J. Liu, M. Li, Z. Li, D. Zhang, C. Zhou, J. Mao, T. Xia, J. Guo, and S. Liu, "A survey of context engineering for large language models," arXiv:2507.13334 [cs.CL], Jul. 2025. [Online]. Available: https://doi.org/10.48550/arxiv.2507.13334